

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ПРЕВРЪЩАНЕ НА ВЪПРОСИ НА ЕСТЕСТВЕН
ЕЗИК ВЪВ ФОРМАЛНИ ЗАЯВКИ:
МОРФОЛОГИЧЕН, СИНТАКТИЧЕН И
СЕМАНТИЧЕН АНАЛИЗ СЪС СИНТЕЗ НА
ПОТВЪРЖДАВАЩО ИЗРЕЧЕНИЕ**

КУРСОВА РАБОТА

по дисциплина „Интерфейс на естествен език“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р Диана Григорова

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Характеристики и особености на естествените езици	3
1.2. Математически модели на естествения език	3
1.3. Психологически модели на езиковата обработка	4
1.4. Модели за представяне на знания и съпоставителна оценка	4
1.5. Интерфейси на естествен език към бази от данни	4
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	6
2.1. Критерии за избор	6
2.2. Морфологичен слой: преобразувател срещу речник от словоформи	6
2.3. Синтактичен слой: Earley срещу СУК	6
2.4. Модел за представяне на знания	7
2.5. Символно решение срещу базово решение с голям езиков модел	8
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	9
3.1. Структура на интерфейса и изисквания към него	9
3.2. Морфологичен анализ	10
3.3. Синтактичен анализ	11
3.4. Семантичен анализ	12
3.5. Генериране на формална заявка	13
3.6. Синтез на изречение	13
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	15
4.1. Обща структура на програмата	15
4.2. Реализация на морфологичния преобразувател	16
4.3. Реализация на синтактичния анализатор	17
4.4. Реализация на семантичния слой	18
4.5. Реализация на генераторите и на синтеза	20
4.6. Демонстрация и мерене	20
4.7. Тестове	21

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	22
5.1. Цел на експеримента	22
5.2. Набор от въпроси	22
5.3. Мерки	22
5.4. Отделен експеримент: нееднозначност спрямо размера на граматиката	23
5.5. Среда на измерването	23
5.6. Съпоставка с базово решение	24
5.7. Условия за валидност	24
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	25
6.1. Условия на измерването	25
6.2. Резултати по слоеве	26
6.3. Поведение при нееднозначност	27
6.4. Нееднозначност спрямо размера на граматиката	28
6.5. Време за анализ	29
6.6. Съпоставка с базовото решение	30
6.7. Анализ	30
ЗАКЛЮЧЕНИЕ	31
ЛИТЕРАТУРА	33
ПРИЛОЖЕНИЯ	35
Приложение А. Формат на лексикалния запис	35
Приложение Б. Граматика на предметната област	37
Приложение В. Модел на знанието	39
Приложение Г. Набор от въпроси и проверка на SQL	42
Приложение Д. Записани измервания	44
Приложение Е. Избрани откъси от изходния текст	46

УВОД

Достъпът до структурирани данни изисква от потребителя да владее формален език за заявки. Това е бариера, която няма нищо общо със самия въпрос, който потребителят иска да зададе: човек, който знае какво търси в една база от данни, често не знае как да го напише на SQL. Интерфейсът на естествен език премахва точно тази бариера, като приема въпроса така, както той се формулира в обикновената реч, и произвежда формалната заявка вместо потребителя.

Настоящата курсова работа разработва такъв интерфейс под името *Lexicon*. Системата приема въпрос на естествен език, анализира го морфологично, синтактично и семантично, и генерира заявка на SQL или SPARQL срещу зададена предметна област. Обработката е изцяло символна: всяка стъпка от входното изречение до изходната заявка е представима, проверима и обяснима, което е съществено различие спрямо решенията, основани на статистически езиков модел.

Целта на проекта е да се изгради работеща верига от анализ и синтез, която превръща въпрос на естествен език във формална заявка и връща на потребителя перифраза на разбрания въпрос, за да може той да потвърди или отхвърли направеното тълкуване, преди заявката да бъде изпълнена.

За постигането на целта са формулирани следните **задачи**:

1. преглед на характеристиките на естествените езици, на математическите и психологическите им модели, и на моделите за представяне на знания, заедно със съпоставителна оценка между тях;
2. анализ на възможните решения за всеки слой от системата, лексикон и морфологичен анализатор, синтактичен анализатор, семантичен слой и генератор на код, и обосновка на направения избор;
3. проектиране на структурата на интерфейса и на договорите между слоевете;
4. програмна реализация на морфологичен анализатор върху краен преобразувател, синтактичен анализатор, работещ при нееднозначност, семантично изображение към модел за представяне на знания и генератор на формална заявка;
5. реализация на синтез на изречение, който перифразира изтълкувания въпрос;
6. изграждане на методика за експериментална проверка върху набор от въпроси и

съпоставка с базово решение, основано на голям езиков модел.

Обхватът на работата е ограничен до въпросителни изречения от предметната област, описана в лексикона и в модела на знанието. Извън обхвата остават свободен диалог, разрешаване на анафора през няколко реплики и произволна предметна област без предварително описание.

Структура на изложението. Раздел I разглежда предметната област и литературата. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката за експериментална проверка, а Раздел VI получените резултати. Заключението обобщава постигнатото и очертава посоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

1.1. Характеристики и особености на естествените езици

Естественият език се различава от формалните езици по няколко свойства, всяко от които е пряка причина за инженерна трудност. Първото е *нееднозначността*: една и съща поредица от думи допуска повече от едно разчитане на всяко ниво, лексикално, морфологично, синтактично и семантично. Второто е *продуктивността*: множеството от допустими изречения е безкрайно, така че системата не може да работи със списък от разпознати въпроси. Третото е *контекстната зависимост*: значението на дадено изречение се променя от предходния текст и от предметната област.

Към тези свойства се добавят особеностите на българския език, който е силно флективен. Една лексема поражда десетки словоформи, а членуването, родът и числото носят синтактична информация, която в английския се изразява с ред на думите или със служебни думи. Това измества тежестта на анализа към морфологичното ниво: без надежден морфологичен анализатор синтактичният анализатор получава вход, в който почти всяка дума е неразпозната.

1.2. Математически модели на естествения език

Математическото описание на езика започва от йерархията на формалните граматик и от въпроса къде в нея попада естественият език. Класическата постановка е дадена в [1] и въвежда генеративната граматика като модел на езиковата компетентност. За целите на инженерната реализация практическият избор почти винаги е безконтекстна граматика, разширена с признаци, защото за нея са известни ефективни алгоритми за анализ [2, 3], а изразителността ѝ покрива преобладаващата част от въпросителните конструкции.

На морфологично ниво съответният математически апарат е този на крайните автомати и на крайните преобразуватели. Преобразувателят изобразява повърхнинна словоформа върху лексикална форма заедно с граматични признаци и обратно, като едно и също устройство служи както за анализ, така и за синтез [4, 5]. Това двупосочно свойство е причината същият апарат да покрие и задачата за синтез на изречение в Раздел III.

1.3. Психологически модели на езиковата обработка

Психолингвистичните модели описват как човек обработва изречение в реално време и имат пряко отражение върху избора на алгоритъм. Инкременталността, тоест изграждането на частична интерпретация още преди изречението да е завършило, е наблюдавана при човешкия анализ и е свойство и на алгоритъма на Earley [2]. Мрежите с преходи [6] са ранен опит да се съчетае изчислителен модел с психологическа правдоподобност. **[TODO: допълнителен източник за психологическите модели на разбиране на изречение, сверен по издателска страница]**

1.4. Модели за представяне на знания и съпоставителна оценка

Формалната заявка се произвежда срещу някакъв модел на знанието, така че изборът на този модел определя какво изобщо може да бъде попитано. Основните семейства са логиката от първи ред, семантичните мрежи и концептуалните графи, фреймовите представяния, релационният модел и графовите модели от типа RDF [7, 8]. Те се различават по изразителност, по цената на извода и по това колко естествено върху тях се изобразява резултатът от семантичния анализ. Съпоставителната оценка по тези критерии е дадена в Раздел II, където се обосновава и направеният избор.

1.5. Интерфейси на естествен език към бази от данни

Задачата има дълга история като самостоятелно направление. Обзорът в [9] систематизира архитектурите на символните системи и описва повтарящите се проблеми, преносимост към нова предметна област, обработка на нееднозначност и обратна връзка към потребителя. По-новите набори от данни и мерки за оценка [10, 11] дават общ терен, върху който символни и статистически решения могат да се сравняват. Обобщение на прегледаните подходи и позиционирането на настоящата разработка спрямо тях е дадено в Табл. 1.

Таблица 1. Прегледани подходи и позициониране на настоящата разработка

Подход	Метод	Нееднозначност	Обяснимост
мрежи с преходи [6]	символен, процедурен	чрез връщане назад в мрежата	следата е мрежата, четима, но процедурна
обзор на символните интерфейси [9]	символен, слоест	разрешава се на семантично ниво	по построение
индуктивно логическо програмиране [10]	обучение върху двойки	статистическа	частична, чрез научените правила
съвременни набори за текст към SQL [11]	статистически	неявна	липсва проверима следа
настоящата разработка	символен, слоест	уплътнена гора и типова проверка	следа от думата до заявката, с причина за отказа

Отделно като свързана работа стои изследване на автора върху съпоставянето с регулярни изрази, което ползва същия апарат от крайни автомати като морфологичния слой тук. **[TODO: цитат на статията за съпоставяне с регулярни изрази]**

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Критерии за избор

Решенията се оценяват по пет критерия: покритие на конструкциите от предметната област, поведение при нееднозначност, обяснимост на резултата, цена на пренасяне към нова предметна област и сложност на реализацията в рамките на курсовата работа. Обяснимостта се извежда като отделен критерий, защото системата връща на потребителя перифраза на разбрания въпрос, а такава перифраза е възможна само ако тълкуването е достъпно като структура, а не само като краен низ.

2.2. Морфологичен слой: преобразувател срещу речник от словоформи

Първият вариант е пълен списък от словоформи с приписани признаци. Той е тривиален за реализация и бърз при търсене, но расте линейно с лексикона и не обработва дума, която не е в списъка. Вторият вариант е краен преобразувател над лексикон и правила за словоизменение [5]. Той изисква повече работа в началото, но е компактен, работи в двете посоки и позволява анализ на непозната дума по нейното окончание. Изборът е вторият вариант, защото синтезът на изречение в Раздел III се нуждае точно от обратната посока на същото устройство.

2.3. Синтактичен слой: Earley срещу CYK

Двата разгледани алгоритъма за анализ при нееднозначност са CYK [3] и Earley [2]. CYK е по-прост за реализация и има предсказуемо поведение, но изисква граматиката да бъде приведена в нормална форма на Чомски, което изкривява дървото на извода и усложнява семантичното изображение. Earley работи с произволна безконтекстна граматика, обработва изречението инкрементално и дава същата асимптотична граница в общия случай, като е по-бърз при еднозначни фрагменти.

Изборът е Earley, и решаващият довод е второто изречение по-горе: нормалната форма на Чомски променя дървото на извода, а семантичното изображение в тази разработка е композиционно и е закачено за продукциите на граматиката такива, каквито

са написани. СУК не е реализиран, затова между двата няма измерена съпоставка по време, и такава не се привежда. Изходът на слоя при всички положения е компактна гора от изводи, а не едно дърво, защото изборът между разчитанията се прави едва след семантичната проверка.

2.4. Модел за представяне на знания

Съпоставката между релационен модел, фреймово представяне и графов модел от типа RDF се прави по критериите от подраздел 2.1 и е дадена в Табл. 2. Релационният модел е близък до целевата заявка на SQL и не изисква допълнително изображение, но описва слабо йерархията от понятия. Графовият модел описва добре йерархията и се запитва със SPARQL, но налага допълнителен слой между семантиката и данните.

Таблица 2. Съпоставителна оценка на моделите за представяне на знания

Критерий	Релационен	Фреймов	Графов, RDF
близост до целевата заявка	пряка, SQL е самият модел	непряка, нужен е превод	пряка, SPARQL е самият модел
йерархия от понятия	слаба, изразява се с допълнителни таблици	силна, наследяването е вградено	силна, изразява се с тройки
изразителност при извод	без вграден извод	извод по подразбиране	извод по схема, извън обхвата тук
цена на пренасяне	нова схема на база	ново описание на фреймове	нов речник от свойства
сложност на реализацията	ниска	средна	ниска, ако се пише текст на заявка

Взетото решение е междинно и е причината логическата форма да съществува като отделен слой. Семантичният слой изобразява върху малка типизирана алгебра от заявки, а не направо върху релационния или върху графовия модел, и от нея излизат и двата целеви езика. Така разликата между тях може да бъде показана, а не постулирана: една и съща логическа форма поражда и SQL, и SPARQL, а типовете и релациите се описват в един файл със схема.

2.5. Символно решение срещу базово решение с голям езиков модел

Голям езиков модел произвежда SQL от въпрос без нито един от изброените слоеве. Тази алтернатива е разгледана изрично, защото е очевидната съвременна съперница на символния подход. Тя има по-широко покритие и не изисква лексикон, но има три съществени недостатъка за настоящата задача: не дава проверима следа от входа до изхода, не може да обоснове защо е избрала едно разчитане пред друго, и се проваля по начин, който потребителят не може да забележи, тъй като изходът изглежда правдоподобен и когато е грешен.

Точно този контраст прави съпоставката интересна. Символната система има по-тясно покритие, но всяко нейно тълкуване е представимо, обяснимо и потвърдимо от потребителя преди изпълнение. Затова езиковият модел се използва тук като *базово решение* за сравнение върху един и същ набор от въпроси, а не като част от системата. Начинът, по който съпоставката се провежда, е описан в Раздел V.

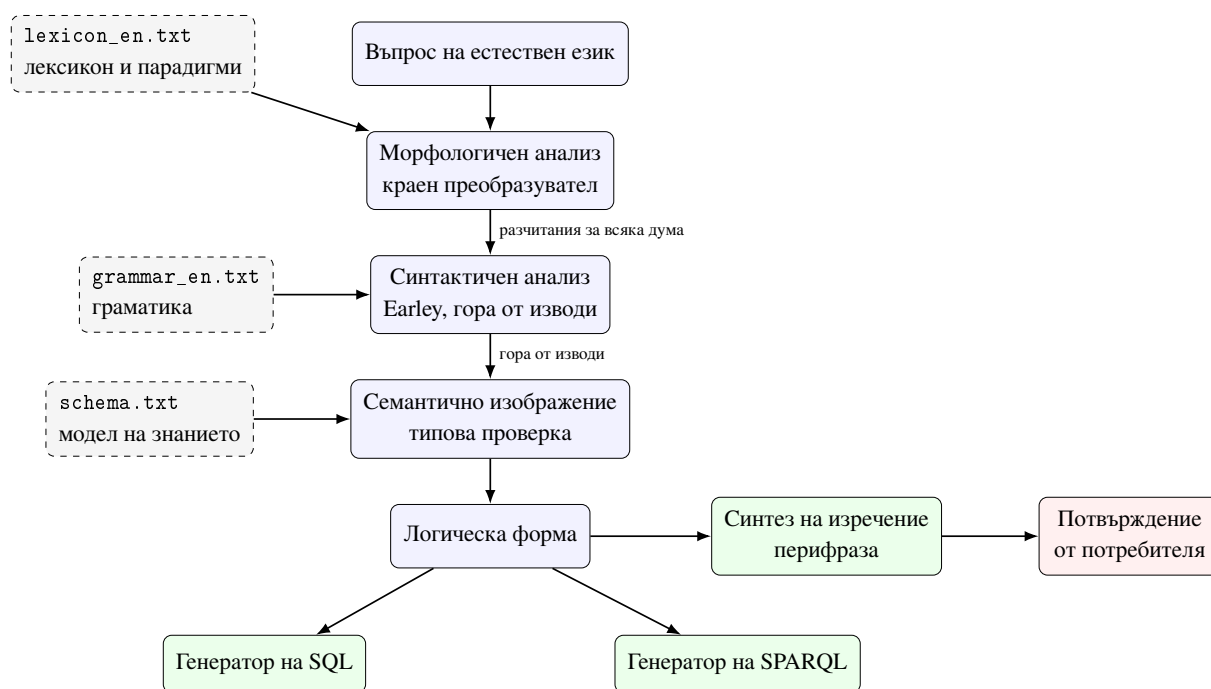
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Структура на интерфейса и изисквания към него

Интерфейсът на естествен език е конвейер от четири анализиращи слоя и един синтезиращ. Входът е низ на естествен език, изходът е формална заявка заедно с перифраза на разбираня въпрос. Слоеве са морфологичен анализ, синтактичен анализ, семантично изображение и генериране на код, а синтезът стои встрани от конвейера и се захранва от семантичното представяне. Структурата и потокът на данните са показани на Фиг. 1.

Изискванията към интерфейса са четири. Всеки слой приема и връща данни по описан договор, така че да може да бъде заменен без промяна в останалите. Нееднозначността се пренася нагоре по конвейера, вместо да се разрешава преждевременно чрез евристика на ниско ниво. Всяко тълкуване е проследимо назад до конкретните думи, които са го породили. Заявката не се изпълнява, преди потребителят да е потвърдил перифразата.

Към тези изисквания е добавено и пето, което произтича от предназначението на разработката: цялото знание за езика и за предметната област живее във файлове с данни, а не в изходния текст. Граматиката не споменава нито една дума, лексиконът не споменава нито една таблица, а генераторите на код не споменават нито една английска конструкция. Заради това разделение смяната на предметната област е смяна на данни.



Фигура 1. Структура на конвейера и поток на данните между слоевете

3.2. Морфологичен анализ

Морфологичният слой е краен преобразувател над лексикона. За всяка входна словоформа той връща множество от разчитания, всяко от които съдържа лексема, част на речта и граматични признаци. Множеството е от разчитания, а не единично разчитане, защото омонимията между словоформи е обичайна и разрешаването ѝ е възможно едва на синтактично ниво. Формата *films* например се анализира и като множествено число на съществителното, и като лична форма на глагола, и двете разчитания се предават нагоре.

Преобразувателят се получава като композиция на две релации. Първата е лексиконът: множество от основи, всяка означена с лексема, част на речта и парадигма. Втората е афиксната релация на парадигмата: списък от форми във вида „изтрий n знака от основата, после добави този суфикс“ заедно с признаците, които афиксът внася. Композицията се извършва еднократно при зареждане, а резултатът е детерминиран ацикличен преобразувател, в който всеки път изписва една повърхнинна словоформа. Същата композиция се пази и в обратната посока, така че синтезът в подраздел 3.6 ползва един и същ ресурс с анализа.

Форматът на лексикалния запис и наборът от граматични признаци са дадени в Табл. 3. Изтриването на знаци в афикса, а не на байтове, е решението, което позволява един и същ механизъм да обслужи и българския лексикон, където една буква заема два

байта в UTF-8.

Таблица 3. Формат на лексикалния запис и набор от граматични признаци

Директива	Полета	Предназначение
PARADIGM	име	открива парадигма
FORM	афикс, признаци	една форма на парадигмата; афиксът е +, +s или ~1+ies
LEX	основа, лексема, част на речта, парадигма, семантично поле	една лексема
Признаци, използвани от парадигмите		
num	sg, pl	число
vform	base, fin, past, part, ger	форма на глагола
tense	pres, past	време
pers	3	лице
def	yes	членуване, само в българския лексикон
Семантични полета на лексикалния запис		
type	име на тип	съществителното означава обекти от този тип
rel	име на релация	глаголът означава тази релация
objprep	предлог	допълнението на глагола идва през този предлог
ent	идентификатор	собственото име означава този обект
prep	предлог	служебна дума, разрешава се през рамките
det	exists, forall, none	квантор, внасян от детерминатора

3.3. Синтактичен анализ

Синтактичният слой приема поредица от множества с разчитания и връща гора от изводи спрямо граматиката на предметната област. Терминал на граматиката не е дума, а част на речта заедно с незадължително ограничение върху признаците, затова граматиката остава независима от лексикона и расте отделно от него.

Изборът на алгоритъм на Earley определя и представянето на резултата. Елементите на таблицата се обединяват по ключ „продукция, позиция на точката, начало“ за всяка

крайна позиция, а всеки елемент носи списък от връзки. Една връзка е един начин да се стигне до този елемент: предшественикът с точка една позиция вляво и възелът от гората, който запълва символа между тях. Нееднозначното изречение затова умножава връзки, а не елементи, и точно това прави гората уплътнена, вместо да е списък от дървета. Възелът от гората се идентифицира с тройката „символ, начало, край“, така че еднаквите подфрази се споделят между разчитанията, вместо да се копират.

Броят на изводите се пресмята върху самата гора чрез динамично оптимизиране: броят за един възел е сумата от броевете на завършените елементи в него, а броят за един елемент е сумата по връзките от произведението на предшественика и на детето. Изброяването на дърветата е отделно извикване с изричен лимит, защото броят на дърветата може да расте много по-бързо от гората, която ги представя.

Нееднозначността по прикрепване живее в точно две продукции на граматиката, $NOM \rightarrow NOM\ PP$ и $VP \rightarrow VP\ PP$. Нищо друго в граматиката не поражда две дървета за един и същ низ, така че отчетената нееднозначност е нееднозначност по прикрепване, а не артефакт от излишна продукция. Граматиката няма празни продукции и няма цикъл от единични продукции, откъдето следва, че гората е ациклична и броят на изводите е краен.

3.4. Семантичен анализ

Семантичният слой изобразява всяко синтактично дърво върху израз в избраното представяне на знания. Изображението е композиционно: всяка продукция на граматиката носи име на семантично правило, а правилото изгражда значението на групата от значенията на съставките ѝ.

Слоят е и мястото, където разчитанията отпадат. Всеки построен атом се проверява типово спрямо модела на знанието, а предлогът се превръща в релация само ако моделът съдържа рамка за този предлог с тези два типа. Рамката е тройката „предлог, тип на аргумента, тип на носителя“ заедно с релацията и реда на аргументите. Затова изразът *in 2010* се прикрепя към филм, а *in Inception* към лице, и двата се различават по типа на именната група, а не по правило за думата *in*. Разчитане, което не се изобразява върху нищо в модела, се отхвърля с посочена причина, и точно това отхвърляне, а не ранна евристика, разрешава нееднозначността.

Когато след типовата проверка оцелее повече от едно разчитане, изборът се прави по предпочитание за близко прикрепване: сумата от разстоянията между всяка предложна

група и главата, към която се прикрепя, измерена в думи. По-малката сума печели. Предпочитанието се прилага след семантичния филтър, никога преди него. Разчитанията, чиито логически форми съвпадат след канонично преименуване на променливите, се събират в едно, така че броят на разчитанията, показан на потребителя, е брой на значенията, а не брой на дърветата.

3.5. Генериране на формална заявка

Генераторът превежда логическата форма в SQL или в SPARQL. Той не съдържа евристики върху текста: работи изцяло върху вече проверената логическа форма, което е причината една и съща семантика да може да се насочи към две различни цели без промяна в предходните слоеве. Съответствието между конструкциите е дадено в Табл. 4.

Таблица 4. Съответствие между логическата форма и целевите езици за заявки

Логическа форма	SQL	SPARQL
едноместен атом Film(x)	FROM film AS t0	?x0 a :Film .
двуместен атом, релация-таблица	FROM directed AS r1 и две равенства	една тройка
двуместен атом, релация-колона	равенство върху колона на същата таблица	една тройка
константа-обект	e0.id = '...'	:p_nolan
проекция	SELECT DISTINCT върху колоната-етикет	SELECT DISTINCT ?x0
броене	SELECT COUNT(DISTINCT ...)	COUNT(DISTINCT ?x0)
въпрос с отговор да или не	SELECT EXISTS (...)	ASK
отрицание	NOT EXISTS (...)	FILTER NOT EXISTS
всеобщност	релационно деление, две вложени NOT EXISTS	две вложени FILTER NOT EXISTS

3.6. Синтез на изречение

Синтезът произвежда изречение на естествен език от семантичното представяне и го показва на потребителя като перифраза на разбрания въпрос. Ролята му е контролна: ако системата е избрала грешно разчитане, потребителят вижда това преди изпълнението на заявката. Затова перифразата се изгражда от логическата форма, а не от въпроса: ако

се изграждаше от въпроса, тя щеше да се съгласи с потребителя по построение и нямаше да потвърждава нищо.

Реализацията ползва обратната посока на морфологичния преобразувател за словоизменението, тоест същия ресурс, който обслужва и анализа. Релацията се изказва с глагола, който я означава в лексикона, или с предлога от съответната рамка, ако лексиконът няма глагол за нея. Съседните определения към една глава се разделят с „и“, което е грубо на слух, но прави двете разчитания на един и същ въпрос две различни изречения, а без това потвърждението от потребителя не носи информация.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Обща структура на програмата

Реализацията е на C++20 и се изгражда с CMake. Разделението на директории следва слоевете от Раздел III: публичните договори са в `include/lexicon/`, реализацията е в `src/`, лексиконът, граматиката, схемата и базата от знания са в `data/`, а тестовете са в `tests/`. Всеки слой е отделна единица за превод със собствен заглавен файл, така че зависимостите между слоевете са видими в списъка с включвания и не могат да се образуват мълчаливо. Съответствието между слой и файлове е дадено в Табл. 5.

Външни зависимости няма. Това е решение, а не пропуск: хранилището е публично и изискването е чужд човек да може да изпълни `cmake -S . -B build` и `cmake --build build` на чиста машина. Затова изпълнителят на тестовете е в самото хранилище, `tests/check.cpp`, времената се мерят със `std::chrono::steady_clock`, а базата от знания е текстов файл, четен при стартиране. При превод с MinGW двете изпълними програми се свързват статично със стандартната библиотека, защото друга инсталация на средата, попаднала по-рано в пътя, иначе подменя динамичната библиотека и процесът пада преди `main`.

Таблица 5. Слоевете, файловете и основни типове

Слой	Файлове	Основни типове
признаци и вход-изход	<code>features.{hpp,cpp}</code>	Features
морфология	<code>morph.{hpp,cpp}</code>	Morphology, LexEntry, Reading
граматика	<code>grammar.{hpp,cpp}</code>	Grammar, Rule, TerminalSpec
синтактичен анализ	<code>earley.{hpp,cpp}</code>	Parser, ParseResult, Item, ForestNode, Tree
логическа форма	<code>logic.{hpp,cpp}</code>	Term, Atom, Formula, Query
семантика	<code>semantics.{hpp,cpp}</code>	Schema, PrepFrame, Interpreter
генератори	<code>codegen.{hpp,cpp}</code>	<code>to_sql</code> , <code>to_sparql</code>
синтез	<code>synth.{hpp,cpp}</code>	paraphrase
знание и извод	<code>kb.{hpp,cpp}</code>	KnowledgeBase, Answer, evaluate
конвейер	<code>pipeline.{hpp,cpp}</code>	System, Analysis, ReadingRecord

4.2. Реализация на морфологичния преобразувател

Преобразувателят се изгражда еднократно от лексикона при стартиране и се запитва за всяка входна словоформа. Състоянията се пазят като масив от структури с ребра по байтове и списък от изходи, а изграждането е композиция на лексикона с афиксната релация на съответната парадигма, показана в Листинг 1. Всеки път през автомата изписва една повърхнинна словоформа, а изходите на приемащото състояние са нейните разчитания.

```
1 // Composition of the lexicon with the affix relation of each paradigm.
2 for (std::size_t index = 0; index < morph.entries_.size(); ++index) {
3     const LexEntry& entry = morph.entries_[index];
4     const auto found = paradigms.find(entry.paradigm);
5     if (found == paradigms.end())
6         throw std::runtime_error("lexeme " + entry.lemma + " uses unknown
↪ paradigm " +
7                                     entry.paradigm);
8     for (const ParadigmForm& form : found->second) {
9         const std::string surface = apply_affix(entry.stem, form);
10        Reading reading;
11        reading.lemma = entry.lemma;
12        reading.pos = entry.pos;
13        reading.feats = form.feats;
14        reading.entry = static_cast<int>(index);
15        morph.add_form(surface, reading);
16        morph.generation_.push_back(GenForm{entry.lemma, entry.pos, form.
↪ feats, surface});
17    }
18 }
```

Листинг 1: Композиция на лексикона с афиксните релации, `src/morph.cpp`

Две решения в този слой заслужават коментар. Първото е, че изтриването в афикса се брои в знаци, а не в байтове. Английският лексикон не би забелязал разликата, българският би: кирилската буква заема два байта в UTF-8 и побайтово изтриване би разсякло буква наполовина. Второто е, че редуването се описва в данните, а не в кода. Записът ~1+ies казва „махни един знак, добави ies“, а ~6+people превръща person в

people. Така неправилната форма е ред в лексикона, а не изключение в изходния текст, и нов език не изисква нов C++.

Изборът на C++20 се вижда в договорите на този слой: `std::string_view` позволява анализът да работи върху изглед към входния низ без копиране, а `std::optional` изразява отсъствието на форма при генериране, без да го смесва с празен низ.

4.3. Реализация на синтактичния анализатор

Анализаторът поддържа таблица от състояния и я допълва инкрементално за всяка позиция във входа. Елементът се обединява по ключ „продукция, точка, начало“ за дадена крайна позиция, а разклоненията се пазят като връзки към предшественик и към възел от гората. Ключовата част е завършването, Листинг 2: завършеният елемент се записва във възела за своя символ и своя обхват, след което всеки чакащ елемент от началната позиция получава нова връзка.

```
1      // Completion. Productions are never empty, so start < position
↳ holds
2      // and the chart position being read is already closed.
3      const int node = node_for(rule.lhs, start, position, false, -1,
↳ -1);
4      std::vector<int>& node_items = result.nodes_[static_cast<std:::
↳ size_t>(node)].items;
5      if (std::find(node_items.begin(), node_items.end(), item_id) ==
↳ node_items.end())
6          node_items.push_back(item_id);
7
8      const std::vector<int>& waiting = chart[static_cast<std:::size_t>(
↳ start)];
9      for (std::size_t w = 0; w < waiting.size(); ++w) {
10         const int waiting_id = waiting[w];
11         const Item& candidate = result.items_[static_cast<std:::size_t>
↳ >(waiting_id)];
12         const Rule& other = grammar.rules()[static_cast<std:::size_t>(
↳ candidate.rule)];
13         if (static_cast<std:::size_t>(candidate.dot) >= other.rhs.size
↳ ()) continue;
```

```

14         if (other.rhs[static_cast<std::size_t>(candidate.dot)] !=
↪ rule.lhs) continue;
15         add_item(candidate.rule, candidate.dot + 1, candidate.start,
↪ position, true,
16                 Link{waiting_id, node});
17     }
18 }

```

Листинг 2: Завършване и построяване на гората, `src/earley.cpp`

Тук се крие и грешката, която реализацията трябваше да избегне. Един и същ възел от гората може да бъде построен от повече от една продукция, и тогава завършването предлага една и съща връзка два пъти. Ако връзките не се проверяват за повторение, броят на изводите нараства, без изречението да е станало по-нееднозначно. Затова `add_item` добавя връзка само ако двойката „предшественик, дете“ вече не присъства.

Броенето на изводите е обхождане на гората с мемоизация и с насищане на горна граница, а не изброяване. Изброяването е отделно извикване с лимит, тъй като броят на дърветата расте много по-бързо от размера на гората. Заради липсата на празни продукции и на цикли от единични продукции обхождането е върху ацикличен граф и приключва.

4.4. Реализация на семантичния слой

Семантичното изображение е обхождане на дървото отдолу нагоре, при което на всеки възел се прилага правилото, свързано със съответната граматична продукция. Междинната стойност е една структура `Sem` с етикет за вида на групата. Вариантен тип тук би добавил церемония без полза: трите вида групи се комбинират от един и същ малък набор правила.

Прикрепването на предложна група, Листинг 3, е мястото, където разчитанията умират. Търси се рамка за тройката „предлог, тип на аргумента, тип на носителя“ и ако модел на знанието няма такава, разчитането се отхвърля с причина, която достига до потребителя дословно. Отхвърлянето е нормална работа на слоя, не програмна грешка, затова се предава с изключение от отделен тип, което конвейерът улавя и записва като причина, вместо да прекъсва анализа.

```

1 // Turns a prepositional phrase into an atom about 'target', or rejects

```

```

↪ the
2 // reading when the knowledge model has no frame for that combination.
3 void attach(const Sem& pp, Sem& target) {
4     const Sem& arg = pp.complement.at(0);
5     const PrepFrame* frame = schema_.frame(pp.prep, arg.type, target.type
↪ );
6     if (frame == nullptr)
7         throw Rejected("no relation for \"" + pp.prep + "\" between " +
↪ target.type + " and " +
8                             arg.type);
9     Atom atom;
10    atom.relation = frame->relation;
11    if (frame->arg_first) {
12        atom.args = {arg.term, target.term};
13    } else {
14        atom.args = {target.term, arg.term};
15    }
16    target.atoms.insert(target.atoms.end(), arg.atoms.begin(), arg.atoms.
↪ end());
17    target.atoms.push_back(std::move(atom));
18    if (pp.head_token >= 0 && target.head_token >= 0) cost_ += pp.
↪ head_token - target.head_token;
19    absorb_quantifier(arg, target.atoms);
20 }

```

Листинг 3: Прикрепване на предложна група и типова проверка, src/semantics.cpp

В същата функция се натрупва и цената на прикрепването, разликата между позицията на предлога и позицията на главата. Тя се използва по-късно за избор между оцелелите разчитания. Съществено е, че предложна група, прикрепена към глаголната група, ограничава допълнението, но разстоянието ѝ се мери от глагола: така по-ниското прикрепване, вътре в именната група, се оказва предпочитаното, което съвпада с наблюдаваното предпочитание при човешкия анализ.

Кванторите се повдигат след като глаголът е дал своя атом. Именната група с every поражда Forall с ограничение от собствените си атоми и с тяло от атомите, които споменават нейната променлива, а групата с no поражда отрицание на същата конюнкция. Обхватът на кванторите е фиксиран по този ред, а не изчислен, което е

съзнателно ограничение и е отбелязано като такова в Раздел VI.

4.5. Реализация на генераторите и на синтеза

Генераторите са две независими функции над логическата форма и схемата. Нито една от тях не вижда въпроса, дървото или лексикона. Генераторът на SQL поддържа таблица от псевдоними и въвежда нов псевдоним само за променливите, които се появяват на текущото ниво; променлива от обхващащо ниво се цитира, което прави вложеното NOT EXISTS корелирано. Всеобщността се превежда като релационно деление: няма свидетел на ограничението, който да не удовлетворява тялото. Генераторът на SPARQL е по-кратък, защото графовият модел не различава релация-таблица от релация-колона и двете се изписват като тройка.

Проверката, че логическата форма е абстракция, а не преоблечен шаблон, е записана като тест: активният и страдателният залог на един и същ въпрос пораждат побайтово еднакви заявки и на двата целеви езика.

Синтезът изгражда изречението от логическата форма. Съществителното се получава от лексемата, въвежда променливата, а числото му от преобразувателя в посока генериране. Релацията се изказва с глагола, който я означава, или с предлога от съответната рамка. Глагол, чието допълнение идва през предлог, като *act in*, не се използва в страдателен залог, и това се разпознава по наличието на полето *objprer* в лексикалния запис, а не по списък с изключения в кода.

4.6. Демонстрация и мерене

Програмата *lexicon* печата всички етапи за един въпрос или за целия набор: означниците с всичките си разчитания, гората с броя на изводите, всяко разчитане заедно с причината да бъде запазено или отхвърлено, избраното разчитане и защо, логическата форма, двата целеви езика, перифразата и отговора. Програмата *lexicon_bench* произвежда числата от Раздел VI.

Отделно *tools/check_sql.py* изпълнява генерирания SQL върху SQLite и сравнява резултата с този на вградения оценител, който работи направо върху логическата форма. Скриптът ползва само стандартната библиотека на Python и не участва в превода.

4.7. Тестове

Тестовите са 30 отделни случая в пет файла и ползват изпълнител от около 90 реда в `tests/check.cpp`. Всеки случай е регистриран в CTest поотделно, така че провалът назовава случая, а не файла. Покритието по слоеве е: морфология седем случая, синтактичен анализ шест, семантика шест, генератори и синтез пет, база от знания три, конвейер три.

Тестовите проверяват поведение, а не достъпни полета. Проверява се, че `films` дава и двете разчитания, че `countries` се анализира, а `country` не, че броят на изводите за изречение с k предложни групи е $k + 1$, че гората побира всички разчитания в по-малко възли, отколкото им трябва поотделно, че типовата проверка отхвърля точно едно от две разчитания при *films with actors in 2010*, и че наборът от въпроси се покрива в очаквания размер. Последният тест е нарочно чувствителен: ако покритието се промени, той се проваля и числото в Раздел VI трябва да се провери отново.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Цел на експеримента

Експериментът проверява три твърдения. Първото е, че системата превръща вярно въпросите от предметната област във формална заявка. Второто е, че логическата форма е действителна абстракция, а не преоблечен шаблон над въпроса. Третото е, че перифразата, върната на потребителя, съответства на построената заявка, тоест че потвърждението от негова страна е информативно, а не формално.

5.2. Набор от въпроси

Наборът се съставя предварително и се замразява, преди да е проведено каквото и да е измерване. Той се съхранява в `data/questions.txt` и се чете както от демонстрацията, така и от програмата за мерене, за да не се окаже, че системата е проверена върху едно множество, а измерена върху друго.

Наборът съдържа въпроси от предметната област, разпределени по вид на конструкцията, и включва изрично подмножество от нееднозначни въпроси. Съдържа и въпроси, за които предварително се знае, че системата ще се провали: конструкции извън граматиката и собствено име извън лексикона. Те стоят там нарочно, за да има покритието върху какво да се провали. Съставът по категории е даден в Раздел VI, Табл. 7.

5.3. Мерки

Използват се пет мерки.

1. **Покритие на синтактичния анализ:** дял на въпросите, за които граматиката допуска поне един извод.
2. **Покритие от край до край:** дял на въпросите, за които поне едно разчитане се изобразява върху модела на знанието и системата произвежда заявка.
3. **Нееднозначност:** брой изводи в гората, брой разчитания, оцелели след типовата проверка, и брой различни логически форми след каноничното сравнение. Трите числа се отчитат поотделно, защото разликите между тях са точно приносът на семантичния слой.

4. **Съгласуваност на генерирания SQL:** дял на въпросите, за които генерираната заявка, изпълнена върху действителна база, връща същия отговор като оценителя, който работи направо върху логическата форма. Двата пътя са независими: оценителят не вижда SQL, а генераторът не вижда базата.
5. **Време за анализ:** медиана и минимум от повторени измервания на синтактичния анализ за изречения с нарастваща дължина.

Времената се мерят със `std::chrono::steady_clock` около самото извикване на анализатора, без зареждането на лексикона и на граматиката, които се извършват еднократно. Всяко измерване се повтаря нечетен брой пъти и се отчита медианата, защото еднократно измерване на анализ под една милисекунда мери часовника, а не анализатора.

5.4. Отделен експеримент: нееднозначност спрямо размера на граматиката

Освен основния набор се провежда и втори експеримент върху три граматики. Първата е работната граматика на системата. Втората добавя две продукции, които описват същите низове по друг път. Третата добавя и трета такава продукция. Никоя от добавените продукции не разширява покритието върху набора от въпроси. Мери се как се променят броят на изводите и броят на различните логически форми. Целта е да се провери твърдението от Раздел III, че нееднозначността, която системата отчита, идва от прикрепването, а не от излишни продукции.

5.5. Среда на измерването

Измерванията се провеждат на една машина и конфигурацията се записва, за да бъдат възпроизводими:

- операционна система: Windows 11, 12 логически процесора;
- компилатор: g++ 15.2.0, MinGW-w64, x86_64-posix-seh;
- режим на превод: CMAKE_BUILD_TYPE=Release, стандарт C++20;
- система за изграждане: CMake 4.3.2 с генератор Ninja 1.13.2;
- проверка на генерирания SQL: модулът sqlite3 от стандартната библиотека на Python 3.14.

Командите, които произвеждат числата, са:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure
./build/lexicon_bench
python tools/check_sql.py build/lexicon
```

Необработеният изход от последните две команди е записан в `docs/measurements/` и е част от хранилището, така че всяко число в Раздел VI може да бъде проследено до реда, от който идва.

5.6. Съпоставка с базово решение

Предвидена беше съпоставка с базово решение, основано на голям езиков модел, на който се подава същият въпрос и същото описание на схемата. **Тази съпоставка не е проведена** в настоящата разработка. Причината е, че тя изисква фиксиране на конкретен модел, версия и настройки, а без тях резултатът не е възпроизводим и не бива да се привежда като число. Раздел VI отбелязва отсъствието ѝ изрично, вместо да го прикрива. **[TODO: конкретният модел, версията му и настройките, ако съпоставката бъде проведена]**

Качественото различие, което е основната причина за такава съпоставка, остава валидно и без нея и е формулирано в Раздел II: символната система показва причината за всяко свое решение, включително причината да отхвърли едно разчитане.

5.7. Условия за валидност

Измерването се смята за валидно само ако наборът от въпроси не е променен след започване на измерването, ако проверката на SQL се извършва срещу оценител, който не участва в построяването на заявката, и ако едно и също изпълнение върху едни и същи данни възпроизвежда същите числа. Всяко нарушение се отбелязва в Раздел VI заедно с резултата, който то засяга.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

6.1. Условия на измерването

Всички числа в този раздел са получени на 19 август 2026 г. на машина с Windows 11 и 12 логически процесора, при превод с g++ 15.2.0 (MinGW-w64) в режим Release по стандарт C++20, CMake 4.3.2 и Ninja 1.13.2. Необработеният изход на измерващите програми е записан в docs/measurements/bench.txt, docs/measurements/sql_check.txt и docs/measurements/per_question.txt и е част от хранилището.

Размерът на ресурсите, върху които са проведени измерванията, е даден в Табл. 6, а съставът на набора от въпроси в Табл. 7.

Таблица 6. Размер на съставените ресурси

Ресурс	Брой
лексеми в английския лексикон	56
повърхнинни словоформи след композицията	80
състояния на преобразувателя	257
преходи на преобразувателя	256
продукции в граматиката	25
символи в граматиката	23
релации в модела на знанието	6
предложни рамки	8
обекти в базата от знания	22
факти в базата от знания	38

Таблица 7. Състав на набора от въпроси по вид на конструкцията

Вид на конструкцията	Брой
активен залог, въпрос към допълнението	5
активен залог, въпрос към подлога	4
страдателен залог	3
отрицание	2
броене	2
въпрос с отговор да или не	2
квантор <i>every</i> или <i>no</i>	2
глагол с предложно допълнение	2
нарочно извън обхвата	3
общо	25
от тях с поне една предложна група	9
от тях с повече от един извод	6

6.2. Резултати по слоеве

Разбивката по слоеве е дадена в Табл. 8. Целта ѝ е грешката да се отнесе към конкретен слой, вместо да остане обща за системата.

Таблица 8. Покритие и съгласуваност по слоеве

Слой	Мярка	Резултат
морфология	въпроси, в които всяка дума е позната на лексикона	22 от 25
синтактичен анализ	въпроси с поне един извод	23 от 25
семантично изображение	въпроси с поне едно разчитане върху модела	22 от 25
генериране на SQL	заявки, които връщат същото като оценителя	22 от 22
генериране на SPARQL	произведена заявка за всеки построен израз	22 от 22
синтез	произведена перифраза за всеки построен израз	22 от 22

Съгласуваността на SQL е измерена независимо: генерираната заявка се изпълнява върху SQLite, зареден със схемата и данните, произведени от `lexicon --emit-sql-schema`, а резултатът се сравнява с отговора на оценителя, който работи

направо върху логическата форма и не вижда SQL. Двадесет и две от двадесет и две заявки съвпадат, включително отрицанията и релационното деление, което идва от *every*.

Трите въпроса, в които се среща непозната дума, са същите три, които са нарочно извън обхвата: *or*, *that* и *Kubrick*. За тях морфологичният слой не се предава, а връща предположение по най-дългия познат афикс, и точно затова два от тях стигат до синтактичния слой, преди да се провалят там.

Отделна мярка за точност на морфологичния анализ спрямо ръчно проверени разчитания **не е провеждана**. Редът в Табл. 8 отчита само дали думите на въпроса са познати, а не дали приписаните признаци са верни за произволен текст. За набора от въпроси признаците се проверяват косвено, чрез седемте случая за морфология в тестовите, но това не е измерване върху независима извадка.

Тестовите обхващат 30 отделни случая, всеки регистриран поотделно в CTest, и всичките 30 минават.

6.3. Поведение при нееднозначност

Шест въпроса от набора допускат повече от един извод. За тях Табл. 9 дава броя изводи от синтактичния слой, броя разчитания, оцелели след типовата проверка, и броя различни логически форми след каноничното сравнение.

Таблица 9. Разчитания преди и след семантичната проверка

Въпрос	изводи	върху модела	значения
Which actors acted in every film by Villeneuve?	2	2	2
Which actors acted in no film by Nolan?	2	2	2
Which films with actors from France did Nolan direct?	2	2	2
Which actors from France acted in films by Nolan?	2	2	1
Who directed a film with Cotillard in 2010?	3	3	1
Which directors from Canada directed films in 2016?	2	2	1

Двете колони вдясно се разминават по два различни начина, и разминаването е точно приносът на семантичния слой.

При трите въпроса в долната половина на таблицата разчитанията се различават

по дървото, но не по значението: предложната група, прикрепена към глаголната група, ограничава същия обект, който би ограничила и вътре в именната група. Каноничното сравнение на логическите форми ги събира в едно и потребителят вижда едно значение, а не три дървета.

При трите въпроса в горната половина разчитанията са наистина различни. Във *Which films with actors from France did Nolan direct?* държавата може да принадлежи на актьорите или на филмите; моделът има релация и за двете, така че и двете разчитания оцеляват и дават различен отговор. Системата избира по-близкото прикрепване, тоест това към актьорите, и го съобщава заедно с цената на прикрепването.

Отхвърляне по типова проверка не се среща в замразения набор, но механизмът се проверява от тестовете. Върху *Which films with actors in 2010 did Nolan direct?* синтактичният слой дава 2 извода, а семантичният отхвърля единия с причината „no relation for in between Person and Year“: моделът няма релация, която да свърже лице с година. Този въпрос не е част от замразения набор и числото не влиза в Табл. 8.

6.4. Неоднозначност спрямо размера на граматиката

Втората половина на експеримента, Табл. 10, минава същия набор през три граматика. Втората добавя две продукции, а третата още една, и никоя от тях не описва низ, който първата не описва.

Таблица 10. Неоднозначност при три граматика над един и същ набор от въпроси

Граматика	продукции	анализирани	изводи	значения	точки
grammar_en.txt	25	23	30	25	7
grammar_v2.txt	27	23	47	25	19
grammar_v3.txt	28	23	47	25	19

Двете допълнителни продукции на втората граматика увеличават изводите от 30 на 47, тоест с 57 на сто, без да променят нито покритието, нито броя на значенията. Те не носят изразителност, а само разклонения, и разклоненията се плащат на всеки следващ слой. Точките на неоднозначност в гората растат от 7 на 19, което е и мястото, където цената се вижда пряко.

Третата граматика не променя нищо. Добавената продукция за съчетание от две съществителни не се задейства, защото нито един въпрос от набора не поставя две

съществителни едно до друго. Цената на една продукция следователно не е самото ѝ съществуване, а това дали изреченията стигат до нея.

Значенията остават 25 и при трите граматики. Излишните продукции нямат семантично правило, затова разчитанията, минали през тях, отпадат в семантичния слой. Това е последователно с проектното решение от Раздел III, но си струва да се каже без заобикалки: причината тези разчитания да отпаднат е, че за тях няма написано правило, а не че моделът на знанието ги е отхвърлил.

6.5. Време за анализ

Таблица 11 дава медианата и минимума от 201 повторения на синтактичния анализ за изречения, всяко от които добавя по една предложна група към предходното. Зареждането на лексикона и на граматиката е извън измерваното време, тъй като се извършва еднократно.

Таблица 11. Време за анализ спрямо дължината на изречението, 201 повторения

групи	думи	изводи	елементи	възли	медиана, us	минимум, us
0	4	1	48	13	14.6	14.1
1	6	2	66	22	22.7	21.9
2	8	3	84	31	30.0	26.5
3	10	4	102	40	35.3	33.6
4	12	5	120	49	45.4	42.3
5	14	6	138	58	52.8	49.6
6	16	7	156	67	64.7	55.9

Броят на изводите расте като $k + 1$ за k предложни групи, а елементите в таблицата и възлите в гората растат линейно, с точно 18 елемента и 9 възела на добавена група. Времето върви заедно с тях: от 14,6 до 64,7 микросекунди при увеличаване на дължината от 4 на 16 думи. В този диапазон растежът е практически линеен и кубичната горна граница на алгоритъма не се проявява, защото изреченията са къси, а граматиката не поражда плътна таблица.

Средното време за анализ на един въпрос от набора, измерено веднъж за всеки въпрос, е 45,1 микросекунди. Това число е по-шумно от таблицата по-горе, защото не е повторено.

6.6. Съпоставка с базовото решение

Съпоставката с базово решение, основано на голям езиков модел, **не е проведена**. Причината е дадена в подраздел 5.6: без фиксиран модел, версия и настройки резултатът не е възпроизводим, а число, което не може да бъде повторено, не бива да стои в таблица. Тук не се привежда оценка по памет или по очакване.

6.7. Анализ

Провалите не са разпределени равномерно, а са концентрирани в два слоя, и са изброени изцяло:

- *Which films did Nolan direct or produce?* се проваля в синтактичния слой: граматиката няма съчинение;
- *Which films that Nolan directed were released in 2010?* се проваля в синтактичния слой: граматиката няма относително изречение;
- *Which films did Kubrick direct?* стига до семантичния слой и се проваля там: собственото име не е в лексикона и не означава обект от базата.

Разпределението е показателно. Двата пропуска в граматиката са структурни и всеки от тях се поправя с продукцията и със семантично правило към нея. Третият е пропуск в лексикона и се поправя с един ред данни. Нито един провал не идва от логическата форма или от генераторите, което съответства и на 22 от 22 съвпадения при проверката на SQL.

Оттук следват два извода за посоката на разширяване. Първо, покритието на граматиката е по-скъпото ограничение: съчинението и относителното изречение се срещат често, а всяко от тях носи нови разклонения и следователно нова нееднозначност, чиято цена е измерена в Табл. 10. Второ, покритието на лексикона е евтино, но расте линейно с предметната област и няма как да се избегне без пренасяне на имената от самата база от знания.

Три ограничения на реализацията заслужават да бъдат отбелязани заедно с резултатите, защото засягат тълкуването им. Моделът на знанието има един тип Person, така че актьорът и режисьорът са едно и също нещо за системата, и въпрос за актьори връща и режисьори. Обхватът на кванторите е фиксиран по реда на повдигането, а не изчислен, така че въпрос с два квантора няма да получи всичките си разчитания. Перифразата разделя съседните определения с „и“, което ги прави различни, но не звучи естествено.

ЗАКЛЮЧЕНИЕ

Проектът изгражда интерфейс на естествен език, който превръща въпрос от обикновената реч във формална заявка чрез морфологичен, синтактичен и семантичен анализ, и връща на потребителя перифраза на разбирания въпрос, преди заявката да бъде изпълнена. Всички слоеве са символни, което прави всяко тълкуване проследимо назад до думите, които са го породили.

Предимства на решението. Обяснимост на всяка стъпка, включително на отказа: когато разчитане бъде отхвърлено, причината е изречение, което стига до потребителя дословно. Възможност за потвърждение преди изпълнение, при това с перифраза, построена от логическата форма, а не от въпроса. Общ ресурс между анализа и синтеза, тъй като морфологичният преобразувател работи в двете посоки. Едно семантично представяне, от което излизат два различни целеви езика без промяна в предходните слоеве. И липса на външни зависимости, така че изграждането на чиста машина иска само компилатор за C++20 и CMake.

Недостатъци. Покритието е ограничено от лексикона и от граматиката, а пренасянето към нова предметна област изисква ръчна работа по описанието на знанието. Моделът няма подтипове, така че актьор и режисьор са едно и също нещо за системата. Обхватът на кванторите е фиксиран по реда на повдигането, а не изчислен. Перифразата разделя съседните определения с „и“, което ги прави различни, но не звучи естествено.

Резултати. От 25 въпроса в замразения набор 23 получават поне един извод и 22 достигат до заявка. Двадесет и две от двадесет и две генерирани SQL заявки, изпълнени върху SQLite, връщат същото като оценителя, който работи направо върху логическата форма. Шест въпроса допускат повече от един извод; при три от тях разчитанията се различават само по дървото и се събират в едно значение, а при три остават наистина различни значения и системата съобщава кое е избрала и защо. Времето за синтактичен анализ расте от 14,6 до 64,7 микросекунди при увеличаване на изречението от 4 на 16 думи. Две допълнителни продукции, които не разширяват покритието, увеличават броя на изводите с 57 на сто, без да добавят нито едно ново значение. Всичките три провала са изброени в Раздел VI поименно: два в граматиката и един в лексикона, нито един в логическата форма или в генераторите.

Посоки за по-нататъшна работа. Съчинение и относително изречение в

граматиката, което покрива двата структурни провала, но носи и нова нееднозначност с измеримата цена от Табл. 10. Подтипове в модела на знанието, за да се различи актьор от режисьор. Изчисляване на обхвата на кванторите вместо фиксиран ред. Пренасяне на собствените имена направо от базата от знания, за да не се дублира лексиконът. И накрая, съпоставката с базово решение върху голям езиков модел, която в настоящата разработка не е проведена и е отбелязана като непроведена, а не приближена по памет.

ЛИТЕРАТУРА

- [1] N. Chomsky, *Syntactic Structures*. The Hague: Mouton, 1957.
- [2] J. Earley, “An efficient context-free parsing algorithm,” *Communications of the ACM*, vol. 13, no. 2, pp. 94–102, 1970.
- [3] D. H. Younger, “Recognition and parsing of context-free languages in time n^3 ,” *Information and Control*, vol. 10, no. 2, pp. 189–208, 1967.
- [4] M. Mohri, “Finite-state transducers in language and speech processing,” *Computational Linguistics*, vol. 23, no. 2, pp. 269–311, 1997.
- [5] K. R. Beesley and L. Karttunen, *Finite State Morphology*. Stanford, CA: CSLI Publications, 2003.
- [6] W. A. Woods, “Transition network grammars for natural language analysis,” *Communications of the ACM*, vol. 13, no. 10, pp. 591–606, 1970.
- [7] J. F. Sowa, *Knowledge Representation: Logical, Philosophical, and Computational Foundations*. Pacific Grove, CA: Brooks/Cole, 2000.
- [8] W3C SPARQL Working Group, “SPARQL 1.1 query language.” W3C Recommendation, 2013. <https://www.w3.org/TR/sparql11-query/>.
- [9] I. Androutsopoulos, G. D. Ritchie, and P. Thanisch, “Natural language interfaces to databases: An introduction,” *Natural Language Engineering*, vol. 1, no. 1, pp. 29–81, 1995.
- [10] J. M. Zelle and R. J. Mooney, “Learning to parse database queries using inductive logic programming,” in *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI-96)*, pp. 1050–1055, 1996.
- [11] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921, 2018.

- [12] D. Jurafsky and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Upper Saddle River, NJ: Pearson Prentice Hall, 2 ed., 2009.

ПРИЛОЖЕНИЯ

Приложение А. Формат на лексикалния запис

Лексиконът се състои от парадигми и лексеми. Парадигмата е списък от форми, всяка от които се задава с афикс и с признаците, които той внася. Афиксът + означава голата основа, +s добавяне на суфикс, а ~1+ies изтриване на един знак и добавяне на суфикс. Лексемата сочи основа, лексема, част на речта, парадигма и семантично поле. Първите редове на английския лексикон, заедно с целия български, са дадени в Листинг 4 и Листинг 5.

```
1 PARADIGM NOUN_S
2 FORM +          num=sg
3 FORM +s         num=pl
4
5 PARADIGM NOUN_ES
6 FORM +          num=sg
7 FORM +es        num=pl
8
9 PARADIGM NOUN_Y
10 FORM +          num=sg
11 FORM ~1+ies     num=pl
12
13 PARADIGM VERB_REG
14 FORM +          vform=base
15 FORM +s         vform=fin tense=pres num=sg pers=3
16 FORM +ed        vform=past tense=past
17 FORM +ed        vform=part
18 FORM +ing       vform=ger
19
20 PARADIGM VERB_E
21 FORM +          vform=base
22 FORM +s         vform=fin tense=pres num=sg pers=3
23 FORM ~1+ed      vform=past tense=past
24 FORM ~1+ed      vform=part
25 FORM ~1+ing     vform=ger
26
27 # "person" has an irregular plural, which the affix syntax expresses by
```

```

↪ deleting
28 # the whole stem: the alternation is data, not a special case in the code.
29 PARADIGM NOUN_PEOPLE
30 FORM +          num=sg
31 FORM ~6+people  num=pl
32
33 PARADIGM INVAR
34 FORM +          -
35
36 # --- common nouns -----
37 LEX person      person      noun      NOUN_PEOPLE type=Person
38 LEX film        film        noun      NOUN_S      type=Film
39 LEX movie       movie       noun      NOUN_S      type=Film
40 LEX actor       actor       noun      NOUN_S      type=Person
41 LEX actress     actress     noun      NOUN_ES     type=Person
42 LEX director    director    noun      NOUN_S      type=Person
43 LEX studio      studio      noun      NOUN_S      type=Studio
44 LEX country     country     noun      NOUN_Y      type=Country
45
46 # The first verb of a relation is the one the synthesiser uses to say it, so
47 # "direct" is listed before the verb "film".
48 LEX direct      direct      verb      VERB_REG    rel=directed
49 # "films" is also a verb form. The homonymy is real and the morphological
↪ layer
50 # keeps both readings; the parser is what decides between them.
51 LEX film        film        verb      VERB_REG    rel=directed
52 LEX produce     produce     verb      VERB_E      rel=produced
53 LEX act         act         verb      VERB_REG    rel=acted_in objprep=in

```

Листинг 4: Парадигми и част от лексемите, data/lexicon_en.txt

```

1 # Малък български лексикон. Обхватът е нарочно тесен: три парадигми и няколко
2 # лексеми, върху които словоизменението е редовно. Целта е да се покаже, че
3 # преобразувателят не зависи от езика, а само от данните: членуването и число
↪ то
4 # се получават от същия механизъм, който обслужва английския лексикон.
5 #
6 # Изтриването в афикса се брои в знаци, а не в байтове, затова ~1 маха една

```

```

7 # кирилска буква, а не половин.
8
9 PARADIGM BG_NOUN_M
10 FORM +          num=sg
11 FORM +ът        num=sg def=yes
12 FORM +и         num=pl
13 FORM +ите       num=pl def=yes
14
15 PARADIGM BG_NOUN_F
16 FORM +          num=sg
17 FORM +та        num=sg def=yes
18 FORM ~1+и       num=pl
19 FORM ~1+ите     num=pl def=yes
20
21 PARADIGM BG_INVAR
22 FORM +          -
23
24 LEX филм        филм      noun    BG_NOUN_M    type=Film
25 LEX актьор      актьор    noun    BG_NOUN_M    type=Person
26 LEX режисьор    режисьор  noun    BG_NOUN_M    type=Person
27 LEX държава     държава   noun    BG_NOUN_F    type=Country
28 LEX година      година     noun    BG_NOUN_F    type=Year
29 LEX в           в         prep    BG_INVAR     prep=in
30 LEX от          от        prep    BG_INVAR     prep=from

```

Листинг 5: Целият български лексикон, data/lexicon_bg.txt

Приложение Б. Граматика на предметната област

```

1 %start S
2
3 %terminal WH      wh
4 %terminal HOWW    howw
5 %terminal MANY    manyw
6 %terminal N       noun
7 %terminal PN      propn
8 %terminal V       verb    vform=base|past
9 %terminal VPART   verb    vform=part

```

10	%terminal	AUXDO	auxdo	
11	%terminal	AUXBE	auxbe	
12	%terminal	DET	det	
13	%terminal	P	prep	
14	%terminal	NEG	neg	
15	%terminal	NUM	num	
16				
17	S	-> WHQ		@pass
18	S	-> YNQ		@pass
19				
20	WHQ	-> WHNP AUXDO NP VP		@wh_do
21	WHQ	-> WHNP AUXDO NP NEG VP		@wh_do_neg
22	WHQ	-> WHNP VP		@wh_subj
23	WHQ	-> WHNP AUXBE VPPASS		@wh_pass
24	WHQ	-> WHNP AUXBE NEG VPPASS		@wh_pass_neg
25	WHQ	-> HOWMANY AUXDO NP VP		@howmany_do
26	WHQ	-> HOWMANY VP		@howmany_subj
27				
28	YNQ	-> AUXDO NP VP		@yn_do
29				
30	HOWMANY	-> HOWW MANY NOM		@howmany_np
31	WHNP	-> WH		@wh_bare
32	WHNP	-> WH NOM		@wh_nom
33				
34	NOM	-> N		@nom_n
35	NOM	-> NOM PP		@nom_pp
36				
37	NP	-> PN		@np_pn
38	NP	-> NUM		@np_num
39	NP	-> NOM		@np_bare
40	NP	-> DET NOM		@np_det
41				
42	VP	-> V		@vp_v
43	VP	-> V NP		@vp_v_np
44	VP	-> VP PP		@vp_pp
45				
46	VPPASS	-> VPART		@vpass_v
47	VPPASS	-> VPPASS PP		@vpass_pp
48				

Листинг 6: Пълният набор от продукции, data/grammar_en.txt

Приложение В. Модел на знанието

Схемата описва типовете, релациите и предложните рамки. Рамката е това, което превръща предлог в релация, и причината едно разчитане да бъде отхвърлено заради липса на значение, а не заради липса на извод.

```

1 TYPE Film      table=film      key=id label=title iri=Film
2 TYPE Person   table=person    key=id label=name  iri=Person
3 TYPE Country  table=country   key=id label=name  iri=Country
4 TYPE Studio   table=studio    key=id label=name  iri=Studio
5 TYPE Year      scalar=int
6
7 REL directed   args=Person,Film store=link table=directed cols=
  ↪ person_id,film_id iri=directed
8 REL acted_in   args=Person,Film store=link table=acted_in cols=
  ↪ person_id,film_id iri=actedIn
9 REL produced   args=Studio,Film store=link table=produced cols=
  ↪ studio_id,film_id iri=produced
10 REL released   args=Film,Year  store=attr table=film      cols=id,year
  ↪                iri=releasedIn
11 REL person_country args=Person,Country store=attr table=person  cols=id,
  ↪ country_id      iri=comesFrom
12 REL film_country  args=Film,Country store=attr table=film      cols=id,
  ↪ country_id      iri=madeIn
13
14 # "in 2010" restricts a film, "in Inception" restricts a person, and the two
  ↪ are
15 # told apart by the type of the noun phrase, not by a rule about the word "in
  ↪ ".
16 FRAME in   arg=Year   attach=Film   rel=released   order=attach,arg
17 FRAME in   arg=Film   attach=Person rel=acted_in   order=attach,arg
18 FRAME from arg=Country attach=Person rel=person_country order=attach,arg
19 FRAME in   arg=Country attach=Person rel=person_country order=attach,arg
20 FRAME by   arg=Person attach=Film   rel=directed   order=arg,attach

```

```

21 FRAME by    arg=Studio  attach=Film  rel=produced      order=arg,attach
22 FRAME with arg=Person  attach=Film  rel=acted_in     order=arg,attach
23 FRAME from arg=Country attach=Film  rel=film_country  order=attach,arg
24
25 # The order of the frames matters in one place only: the synthesiser says a
26 # relation with the first frame that fits, so "from" is listed before "in"
  ↪ for
27 # the country of a person.

```

Листинг 7: Типове, релации и предложни рамки, data/schema.txt

Базата от знания е малка ръчно написана извадка от каталог на филми, достатъчно голяма, за да дадат различните разчитания на един въпрос различни отговори, и достатъчно малка, за да бъде проверена на око.

```

1
2 ENTITY Film    f_inception    "Inception"
3 ENTITY Film    f_interstellar  "Interstellar"
4 ENTITY Film    f_dunkirk       "Dunkirk"
5 ENTITY Film    f_arrival       "Arrival"
6 ENTITY Film    f_amelie        "Amelie"
7
8 ENTITY Person  p_nolan         "Christopher Nolan"
9 ENTITY Person  p_villeneuve    "Denis Villeneuve"
10 ENTITY Person p_jeunet        "Jean-Pierre Jeunet"
11 ENTITY Person p_dicaprio      "Leonardo DiCaprio"
12 ENTITY Person p_cotillard     "Marion Cotillard"
13 ENTITY Person p_hardy         "Tom Hardy"
14 ENTITY Person p_murphy        "Cillian Murphy"
15 ENTITY Person p_mcconaughey   "Matthew McConaughey"
16 ENTITY Person p_hathaway      "Anne Hathaway"
17 ENTITY Person p_adams         "Amy Adams"
18 ENTITY Person p_tautou        "Audrey Tautou"
19
20 ENTITY Country c_uk            "United Kingdom"
21 ENTITY Country c_usa          "United States"
22 ENTITY Country c_france       "France"
23 ENTITY Country c_canada       "Canada"
24 ENTITY Country c_ireland      "Ireland"

```

```

25
26 ENTITY Studio s_warner "Warner Bros"
27
28 FACT directed p_nolan f_inception
29 FACT directed p_nolan f_interstellar
30 FACT directed p_nolan f_dunkirk
31 FACT directed p_villeneuve f_arrival
32 FACT directed p_jeunet f_amelie
33
34 FACT acted_in p_dicaprio f_inception
35 FACT acted_in p_cotillard f_inception
36 FACT acted_in p_hardy f_inception
37 FACT acted_in p_murphy f_inception
38 FACT acted_in p_hardy f_dunkirk
39 FACT acted_in p_murphy f_dunkirk
40 FACT acted_in p_mcconaughey f_interstellar
41 FACT acted_in p_hathaway f_interstellar
42 FACT acted_in p_adams f_arrival
43 FACT acted_in p_tautou f_amelie
44
45 FACT released f_inception 2010
46 FACT released f_interstellar 2014
47 FACT released f_dunkirk 2017
48 FACT released f_arrival 2016
49 FACT released f_amelie 2001
50
51 FACT person_country p_nolan c_uk
52 FACT person_country p_villeneuve c_canada
53 FACT person_country p_jeunet c_france
54 FACT person_country p_dicaprio c_usa
55 FACT person_country p_cotillard c_france
56 FACT person_country p_hardy c_uk
57 FACT person_country p_murphy c_ireland
58 FACT person_country p_mcconaughey c_usa
59 FACT person_country p_hathaway c_usa
60 FACT person_country p_adams c_usa
61 FACT person_country p_tautou c_france
62
63 FACT film_country f_inception c_usa

```

```

64 FACT film_country f_interstellar c_usa
65 FACT film_country f_dunkirk      c_uk
66 FACT film_country f_arrival      c_usa
67 FACT film_country f_amelie       c_france
68
69 FACT produced s_warner f_inception
70 FACT produced s_warner f_dunkirk

```

Листинг 8: Вградената база от знания, data/kb.txt

Приложение Г. Набор от въпроси и проверка на генерирания SQL

Наборът е замразен преди измерването. Последните три въпроса са нарочно извън обхвата.

```

1 Who directed Inception?
2 What did Nolan direct?
3 Which films did Nolan direct?
4 Which films did Nolan direct in 2010?
5 Which films were directed by Nolan?
6 Which films were not directed by Nolan?
7 Which films did Nolan not direct?
8 How many films did Nolan direct?
9 Did Nolan direct Inception?
10 Did Villeneuve direct Inception?
11 Which actors acted in Inception?
12 How many actors acted in Dunkirk?
13 Which actors acted in every film by Villeneuve?
14 Which actors acted in no film by Nolan?
15 Which films with actors from France did Nolan direct?
16 Which films with Cotillard were directed by Nolan?
17 Which actors from France acted in films by Nolan?
18 Who directed a film with Cotillard in 2010?
19 Which studios produced Dunkirk?
20 Which films did Warner produce?
21 Which films were produced by Warner in 2010?
22 Which directors from Canada directed films in 2016?
23 Which films did Nolan direct or produce?

```

```
24 Which films that Nolan directed were released in 2010?
25 Which films did Kubrick direct?
```

Листинг 9: Замразеният набор от въпроси, data/questions.txt

Вместо еталонни заявки, съставени на ръка, съответствието се проверява чрез изпълнение: генерираният SQL се пуска върху SQLite и резултатът се сравнява с този на оценителя, който работи направо върху логическата форма. Записаният изход е в Листинг 10.

```
1 # Produced by: python tools/check_sql.py build/lexicon
2
3 OK Who directed Inception? -> Christopher Nolan
4 OK What did Nolan direct? -> Dunkirk, Inception, Interstellar
5 OK Which films did Nolan direct? -> Dunkirk, Inception, Interstellar
6 OK Which films did Nolan direct in 2010? -> Inception
7 OK Which films were directed by Nolan? -> Dunkirk, Inception,
  ↪ Interstellar
8 OK Which films were not directed by Nolan? -> Amelie, Arrival
9 OK Which films did Nolan not direct? -> Amelie, Arrival
10 OK How many films did Nolan direct? -> 3
11 OK Did Nolan direct Inception? -> yes
12 OK Did Villeneuve direct Inception? -> no
13 OK Which actors acted in Inception? -> Cillian Murphy, Leonardo DiCaprio,
  ↪ Marion Cotillard, Tom Hardy
14 OK How many actors acted in Dunkirk? -> 2
15 OK Which actors acted in every film by Villeneuve? -> Amy Adams
16 OK Which actors acted in no film by Nolan? -> Amy Adams, Audrey Tautou,
  ↪ Christopher Nolan, Denis Villeneuve, Jean-Pierre Jeunet
17 OK Which films with actors from France did Nolan direct? -> Inception
18 OK Which films with Cotillard were directed by Nolan? -> Inception
19 OK Which actors from France acted in films by Nolan? -> Marion Cotillard
20 OK Who directed a film with Cotillard in 2010? -> Christopher Nolan
21 OK Which studios produced Dunkirk? -> Warner Bros
22 OK Which films did Warner produce? -> Dunkirk, Inception
23 OK Which films were produced by Warner in 2010? -> Inception
24 OK Which directors from Canada directed films in 2016? -> Denis
  ↪ Villeneuve
```

```

25 SKIP Which films did Nolan direct or produce? (no parse: the grammar does
    ↪ not describe this sentence)
26 SKIP Which films that Nolan directed were released in 2010? (no parse: the
    ↪ grammar does not describe this sentence)
27 SKIP Which films did Kubrick direct? (no reading of this sentence maps onto
    ↪ the knowledge model)
28
29 22/22 generated queries agree with the evaluator, 3 question(s) produced no
    ↪ query

```

Листинг 10: Записан резултат от кръстосаната проверка,
docs/measurements/sql_check.txt

Приложение Д. Записани измервания

```

1 # Raw benchmark output. Produced by:
2 #   cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
3 #   cmake --build build
4 #   ./build/lexicon_bench
5 # Machine: Windows 11, 12 logical CPUs, g++ 15.2.0 (MinGW-w64), CMake 4.3.2,
    ↪ Ninja 1.13.2.
6 # Date: 2026-08-19
7
8 Lexicon benchmark. Every number below was produced by this run.
9
10 == Size of the compiled resources ==
11 lexemes                      56
12 surface forms                 80
13 transducer states             257
14 transducer transitions        256
15 grammar productions           25
16 grammar symbols               23
17 relations in the model        6
18 prepositional frames          8
19 entities                      22
20 facts                        38
21

```

```

22 == Parse time against sentence length ==
23 phrases  words  derivations  chart items  forest nodes  median us  best us
24      0      4          1         48         13      14.6    14.1
25      1      6          2         66         22      22.7    21.9
26      2      8          3         84         31      30.0    26.5
27      3     10          4        102         40      35.3    33.6
28      4     12          5        120         49      45.4    42.3
29      5     14          6        138         58      52.8    49.6
30      6     16          7        156         67      64.7    55.9
31
32 == Ambiguity against grammar size ==
33 Three grammars over the same question set. The second and the third add
34 productions that describe nothing the first cannot already describe.
35
36 grammar      rules  parsed  derivations  distinct forms  ambiguity points
37 grammar_en.txt  25    23      30           25              7
38 grammar_v2.txt  27    23      47           25             19
39 grammar_v3.txt  28    23      47           25             19
40
41 == Coverage over the bundled question set ==
42 questions                25
43 parsed                    23
44 answered                  22
45 more than one parse       6
46 mean parse time          45.1 us
47
48 failures, in full:
49   Which films did Nolan direct or produce? -> no parse: the grammar does
    ↪ not describe this sentence
50   Which films that Nolan directed were released in 2010? -> no parse: the
    ↪ grammar does not describe this sentence
51   Which films did Kubrick direct? -> no reading of this sentence maps onto
    ↪ the knowledge model

```

Листинг 11: Необработен изход на измерващата програма,
docs/measurements/bench.txt

Приложение Е. Избрани откъси от изходния текст

Показват се само относимите откъси, а не целите файлове. Всеки откъс се чете от истинския файл при компилация с `\lstinputlisting`, тоест не може да се разминава с хранилището.

```
1 class Morphology {
2 public:
3     static Morphology load(const std::string& path);
4
5     // All analyses of a surface form, in lexicon order. Empty when the form
6     ↪ is
7     // not in the compiled transducer.
8     std::vector<Reading> analyze(std::string_view form) const;
9
10    // Analyses of an unknown form, taken from the longest affix that some
11    // paradigm uses. Called only when analyze() returns nothing.
12    std::vector<Reading> guess(std::string_view form) const;
13
14    // Inverse direction: the surface form of a lemma whose features include
15    ↪ all
16    // the requested ones. Empty when the lexicon has no such form.
17    std::optional<std::string> generate(const std::string& lemma, const std::
18    ↪ string& pos,
19                                     const Features& wanted) const;
20
21    const std::vector<LexEntry>& entries() const { return entries_; }
22    const LexEntry* entry(int index) const;
23
24    // Size of the compiled transducer, reported by the demo and the
25    ↪ benchmark.
26
27    std::size_t state_count() const { return states_.size(); }
28    std::size_t edge_count() const;
29    std::size_t form_count() const { return form_count_; }
30    std::size_t lexeme_count() const { return entries_.size(); }
```

Листинг 12: Договорът на морфологичния слой, `include/lexicon/morph.hpp`

```

1  std::vector<Reading> Morphology::analyze(std::string_view form) const {
2      const std::string key = to_lower_ascii(form);
3      int current = 0;
4      for (const char c : key) {
5          int next = -1;
6          for (const auto& edge : states_[static_cast<std::size_t>(current)].
↪ edges)
7              if (edge.first == c) next = edge.second;
8          if (next < 0) return {};
9          current = next;
10     }
11     std::vector<Reading> result;
12     for (const int output : states_[static_cast<std::size_t>(current)].
↪ outputs)
13         result.push_back(outputs_[static_cast<std::size_t>(output)]);
14     return result;
15 }
16
17 std::vector<Reading> Morphology::guess(std::string_view form) const {
18     const std::string key = to_lower_ascii(form);
19     std::vector<Reading> result;
20     std::size_t best = 0;
21     for (const AffixHint& hint : hints_) {
22         if (hint.suffix.size() > key.size()) continue;
23         if (key.compare(key.size() - hint.suffix.size(), hint.suffix.size(),
↪ hint.suffix) != 0)
24             continue;
25         if (!result.empty() && hint.suffix.size() < best) break;
26         best = hint.suffix.size();
27         Reading reading;
28         reading.lemma = key.substr(0, key.size() - hint.suffix.size());
29         reading.pos = hint.pos;
30         reading.feats = hint.feats;
31         reading.guessed = true;
32         if (std::find(result.begin(), result.end(), reading) == result.end())
33             result.push_back(std::move(reading));
34     }
35     if (result.empty()) {

```

```

36     Reading reading;
37     reading.lemma = key;
38     reading.pos = "noun";
39     reading.feats = Features::parse("num=sg");
40     reading.guessed = true;
41     result.push_back(std::move(reading));
42 }
43 return result;
44 }
45
46 std::optional<std::string> Morphology::generate(const std::string& lemma,
↳ const std::string& pos,
47                                             const Features& wanted) const
↳ {
48     for (const GenForm& form : generation_) {
49         if (form.lemma != lemma) continue;
50         if (!pos.empty() && form.pos != pos) continue;
51         if (!form.feats.satisfies(wanted)) continue;
52         return form.surface;
53     }
54     return std::nullopt;
55 }

```

Листинг 13: Анализ, отгатване и генериране върху един и същ преобразувател, src/morph.cpp

```

1 struct Link {
2     int predecessor = -1; // item one dot to the left, -1 when the dot is at
↳ 0
3     int child = -1;      // forest node filling the symbol
4 };
5
6 struct Item {
7     int rule = 0;
8     int dot = 0;
9     int start = 0;
10    int end = 0;
11    std::vector<Link> links;

```

```

12 };
13
14 struct ForestNode {
15     int symbol = -1;
16     int start = 0;
17     int end = 0;
18     bool terminal = false;
19     int token = -1;           // terminal nodes: index of the token
20     int reading = -1;        // terminal nodes: index of the matching
    ⇨ reading
21     std::vector<int> items;    // non terminal nodes: completed items
    ⇨ deriving it
22 };
23
24 // A single reading extracted from the forest.
25 struct Tree {
26     int symbol = -1;
27     int rule = -1;           // -1 for terminal leaves
28     int token = -1;          // >= 0 for terminal leaves
29     int reading = -1;
30     std::vector<Tree> children;
31 };

```

Листинг 14: Представяне на елемента, на връзката и на гората,
include/lexicon/earley.hpp

```

1     // Derivations are counted on the forest, never by enumerating trees.
2     std::vector<std::uint64_t> item_counts(result.items_.size(), 0);
3     std::vector<char> item_done(result.items_.size(), 0);
4     std::vector<std::uint64_t> node_counts(result.nodes_.size(), 0);
5     std::vector<char> node_done(result.nodes_.size(), 0);
6
7     std::function<std::uint64_t(int)> count_item;
8     std::function<std::uint64_t(int)> count_node;
9
10    count_item = [&](int id) -> std::uint64_t {
11        const std::size_t index = static_cast<std::size_t>(id);
12        if (item_done[index] == 2) return item_counts[index];

```

```

13         if (item_done[index] == 1) return 0; // guard, the grammar has
↪ no cycles
14         item_done[index] = 1;
15         const Item& item = result.items_[index];
16         std::uint64_t total = 0;
17         if (item.dot == 0) {
18             total = 1;
19         } else {
20             for (const Link& link : item.links)
21                 total = saturating_add(total, saturating_mul(count_item(
↪ link.predecessor),
22                                     count_node(
↪ link.child)));
23         }
24         item_counts[index] = total;
25         item_done[index] = 2;
26         return total;
27     };
28     count_node = [&](int id) -> std::uint64_t {
29         const std::size_t index = static_cast<std::size_t>(id);
30         if (node_done[index] == 2) return node_counts[index];
31         if (node_done[index] == 1) return 0;
32         node_done[index] = 1;
33         const ForestNode& node = result.nodes_[index];
34         std::uint64_t total = 0;
35         if (node.terminal) {
36             total = 1;
37         } else {
38             for (const int item : node.items) total = saturating_add(
↪ total, count_item(item));
39         }
40         node_counts[index] = total;
41         node_done[index] = 2;
42         return total;
43     };
44     result.derivations_ = count_node(result.root_);
45 }

```

Листинг 15: Броене на изводите върху гората, без изброяване, src/earley.cpp

```

1 struct Term {
2     enum class Kind { Variable, Entity, Number };
3
4     Kind kind = Kind::Variable;
5     int var = -1;
6     std::string entity;
7     long long number = 0;
8
9     static Term variable(int id);
10    static Term entity_ref(std::string id);
11    static Term numeric(long long value);
12    std::string str() const;
13    bool operator==(const Term& other) const;
14 };
15
16 struct Atom {
17     std::string relation;
18     std::vector<Term> args;
19     std::string str() const;
20 };
21
22 struct Formula;
23 using FormulaRef = std::shared_ptr<const Formula>;
24
25 struct Formula {
26     enum class Kind { Atom, And, Not, Forall };
27
28     Kind kind = Kind::Atom;
29     Atom atom;
30     std::vector<FormulaRef> subs;
31     int var = -1; // Forall: subs[0] restricts this variable, subs[1] is the
    ↪ body
32 };
33
34 FormulaRef make_atom(Atom atom);
35 FormulaRef make_and(std::vector<FormulaRef> parts);
36 FormulaRef make_not(FormulaRef sub);
37 FormulaRef make_forall(int var, FormulaRef restriction, FormulaRef body);

```

```

38 std::string to_string(const FormulaRef& formula);
39
40 // Every variable that occurs anywhere in the formula, in order of appearance
41 ↪ .
42
43 std::vector<int> variables_of(const FormulaRef& formula);
44
45 struct Query {
46     std::vector<int> result_vars;
47     bool count = false;    // "how many"
48     bool boolean = false;  // yes or no question
49     FormulaRef body;
50     std::map<int, std::string> var_types;
51     std::map<int, std::string> var_labels; // the noun that introduced the
52 ↪ variable
53
54     std::string str() const;
55
56     // The same string for two readings that mean the same thing. Variable
57     // numbers are reassigned by order of appearance and the conjuncts are
58     // sorted, so a difference in the parse tree that makes no difference to
59 ↪ the
60
61     // meaning does not survive into the key.
62     std::string canonical() const;
63 };

```

Листинг 16: Логическата форма, include/lexicon/logic.hpp

```

1 Sem assemble(Clauses parts) {
2     const RelationSpec* relation = schema_.relation(parts.verb.relation);
3     if (relation == nullptr || relation->args.size() != 2)
4         throw Rejected("the verb does not denote a binary relation of the
5 ↪ model");
6
7     Sem subject;
8     Sem object;
9     bool have_subject = false;
10    bool have_object = false;

```

```

11     if (parts.passive) {
12         // The by phrase of a passive fills the subject slot rather than
13         // adding a relation of its own.
14         for (std::size_t i = 0; i < parts.verb.pending.size(); ++i) {
15             const Sem& pp = parts.verb.pending[i];
16             if (pp.prep != "by") continue;
17             if (pp.complement.at(0).type != relation->args[0]) continue;
18             subject = pp.complement.at(0);
19             have_subject = true;
20             parts.verb.pending.erase(parts.verb.pending.begin() +
↪ static_cast<long>(i));
21             break;
22         }
23         if (!have_subject) {
24             subject.term = Term::variable(fresh(relation->args[0],
↪ relation->args[0]));
25             subject.type = relation->args[0];
26             have_subject = true;
27         }
28     } else if (!parts.subject.empty()) {
29         subject = parts.subject.front();
30         have_subject = true;
31     }

```

Листинг 17: Сглобяване на изречението и повдигане на кванторите, src/semantics.cpp

```

1 class SparqlWriter {
2 public:
3     SparqlWriter(const Schema& schema, const Query& query) : schema_(schema),
↪ query_(query) {}
4
5     std::string run() const {
6         const std::string patterns = block(query_.body, " ");
7         std::string head;
8         if (query_.boolean) {
9             head = "ASK";
10        } else if (query_.count) {
11            head = "SELECT (COUNT(DISTINCT " + term(Term::variable(query_.

```

```

12 ↪ result_vars.front())) +
        ") AS ?answer)";
13     } else {
14         head = "SELECT DISTINCT " + term(Term::variable(query_.
15 ↪ result_vars.front()));
        }
16         std::string text = "PREFIX : <http://example.org/film#>\n" + head + "
17 ↪ WHERE {\n" + patterns +
        "\n}";
18         if (!query_.boolean && !query_.count)
19             text += "\nORDER BY " + term(Term::variable(query_.result_vars.
20 ↪ front()));
        return text;
21     }
22
23 private:
24     const Schema& schema_;
25     const Query& query_;
26
27     std::string term(const Term& value) const {
28         switch (value.kind) {
29             case Term::Kind::Variable: return "?x" + std::to_string(value.var
30 ↪ );
            case Term::Kind::Entity: return ":" + value.entity;
            case Term::Kind::Number: return std::to_string(value.number);
31         }
32         return "?";
33     }
34
35     std::string block(const FormulaRef& formula, const std::string& indent)
36 ↪ const {
37         std::vector<FormulaRef> parts;
38         flatten(formula, parts);
39         std::string out;
40         for (const FormulaRef& part : parts) {
41             switch (part->kind) {
42                 case Formula::Kind::Atom: {
43                     const Atom& atom = part->atom;
44                     if (atom.args.size() == 1) {

```

```

45         const TypeSpec* spec = schema_.type(atom.relation);
46         out += indent + term(atom.args[0]) + " a :" +
47             (spec != nullptr ? spec->iri : atom.relation)
↳ + " .\n";
48         break;
49     }
50     const RelationSpec* spec = schema_.relation(atom.relation
↳ );
51     if (spec == nullptr)
52         throw std::runtime_error("no relation " + atom.
↳ relation + " in the schema");
53     out += indent + term(atom.args.at(0)) + " :" + spec->iri
↳ + " " +
54         term(atom.args.at(1)) + " .\n";
55     break;
56 }
57 case Formula::Kind::Not:
58     out += indent + "FILTER NOT EXISTS {\n" + block(part->
↳ subs.at(0), indent + " ") +
59         indent + "}\n";
60     break;
61 case Formula::Kind::Forall:
62     out += indent + "FILTER NOT EXISTS {\n" +
63         block(part->subs.at(0), indent + " ") + indent +
64         " FILTER NOT EXISTS {\n" + block(part->subs.at(1)
↳ , indent + " ") +
65         indent + " }\n" + indent + "}\n";
66     break;
67 case Formula::Kind::And:
68     out += block(part, indent);
69     break;
70 }
71 }
72 return out;
73 }
74 };

```

Листинг 18: Генератор на SPARQL, src/codegen.cpp